Experiment 8: Code Changes for Real-Time Communication Using Socket.IO

1. `backend/server.js` — Socket.IO Setup and Announcement API

```
const http = require("http");
const { Server } = require("socket.io");

const server = http.createServer(app);

const io = new Server(server, {
  cors: {
    origin: [
      "http://localhost:8088",
      "http://localhost:5173"
    ],
    methods: ["GET", "POST"]
  }
});

app.post(
  "/api/announcements",
  authenticateToken,
  authorizeRoles("Admin"),
  (req, res) => {
    const { message } = req.body;

    if (typeof message !== "string" || !message.trim()) {
      return res.status(400).json({
        success: false,
        message: "Announcement message is required"
      });
    }

    const announcement = {
      message: message.trim(),
      timestamp: new Date().toISOString()
    };

    io.to("announcements").emit(
      "new-announcement",
      announcement
```

```
    );

    return res.status(201).json({
      success: true,
      message: "Announcement published successfully",
      announcement
    });
  }
);

io.on("connection", (socket) => {
  console.log("Socket connected:", socket.id);

  socket.on("join-announcements", () => {
    socket.join("announcements");
  });

  socket.on("disconnect", () => {
    console.log("Socket disconnected:", socket.id);
  });
});
```

Server Startup Change

```
server.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

2. `frontend/src/App.jsx` — Socket.IO Connection

```
const API_URL = import.meta.env.VITE_API_URL || "";
const SOCKET_URL =
  import.meta.env.VITE_SOCKET_URL || "http://localhost:5000";

useEffect(() => {
  const socketConnection = io(SOCKET_URL);

  socketConnection.on("connect", () => {
    setConnectionStatus("Connected");
    socketConnection.emit("join-announcements");
  });

  socketConnection.on("disconnect", () => {
    setConnectionStatus("Disconnected");
  });
```

```
  socketConnection.on("connect_error", () => {
    setConnectionStatus("Connection error");
  });

  socketConnection.on("new-announcement", (announcement) => {
    setAnnouncements((previous) => [
      announcement,
      ...previous
    ]);
  });

  return () => {
    socketConnection.disconnect();
  };
}, []);
```

3. `frontend/src/App.jsx` — Admin Announcement Publishing

```
const publishAnnouncement = async () => {
  if (!announcementMessage.trim()) {
    setPublishStatus("Please enter an announcement.");
    return;
  }

  setPublishing(true);
  setPublishStatus("");

  try {
    const response = await fetch(
      `${API_URL}/api/announcements`,
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          Authorization: `Bearer ${token}`
        },
        body: JSON.stringify({
          message: announcementMessage
        })
      }
    );

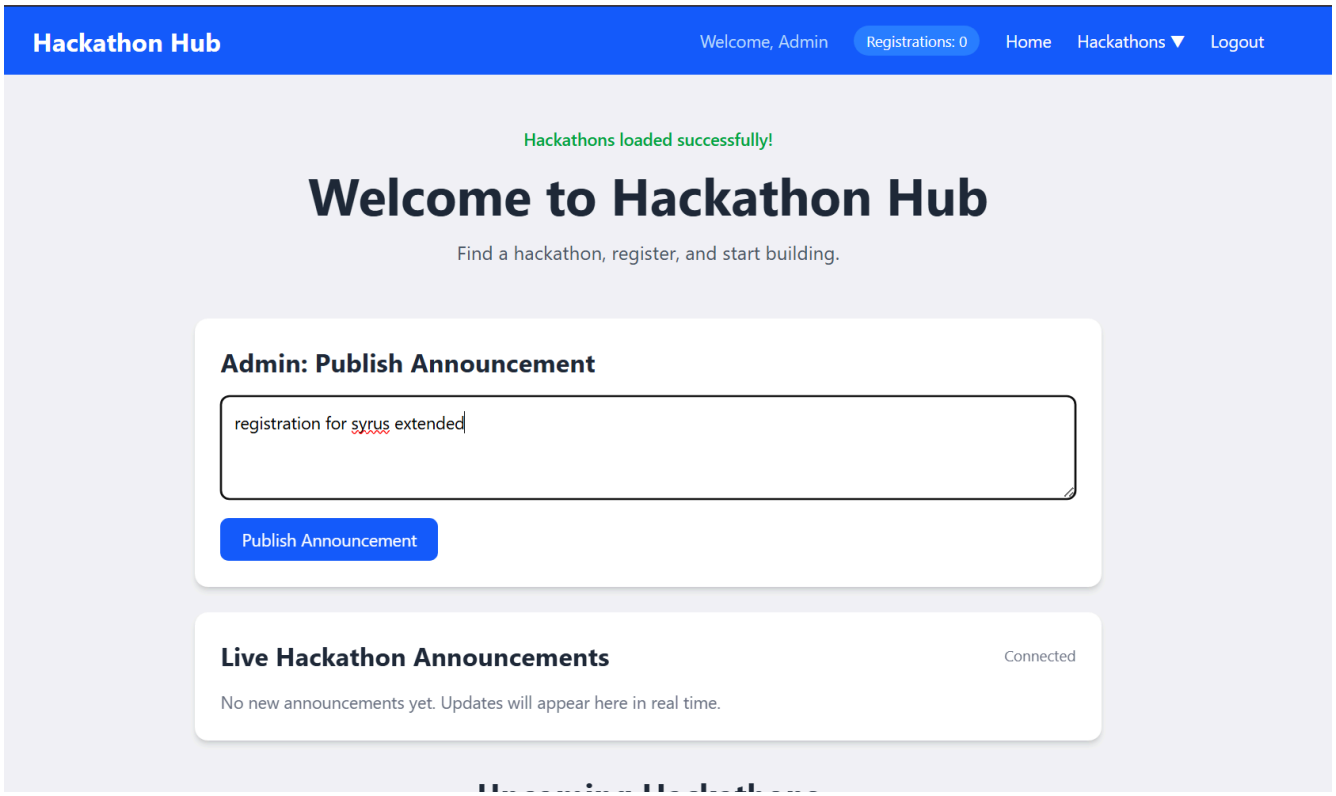
    const data = await response.json();
```

```
    if (!response.ok) {
      throw new Error(
        data.message || "Failed to publish announcement"
      );
    }

    setAnnouncementMessage("");
    setPublishStatus("Announcement published successfully!");
  } catch (error) {
    setPublishStatus(error.message);
  } finally {
    setPublishing(false);
  }
};
```

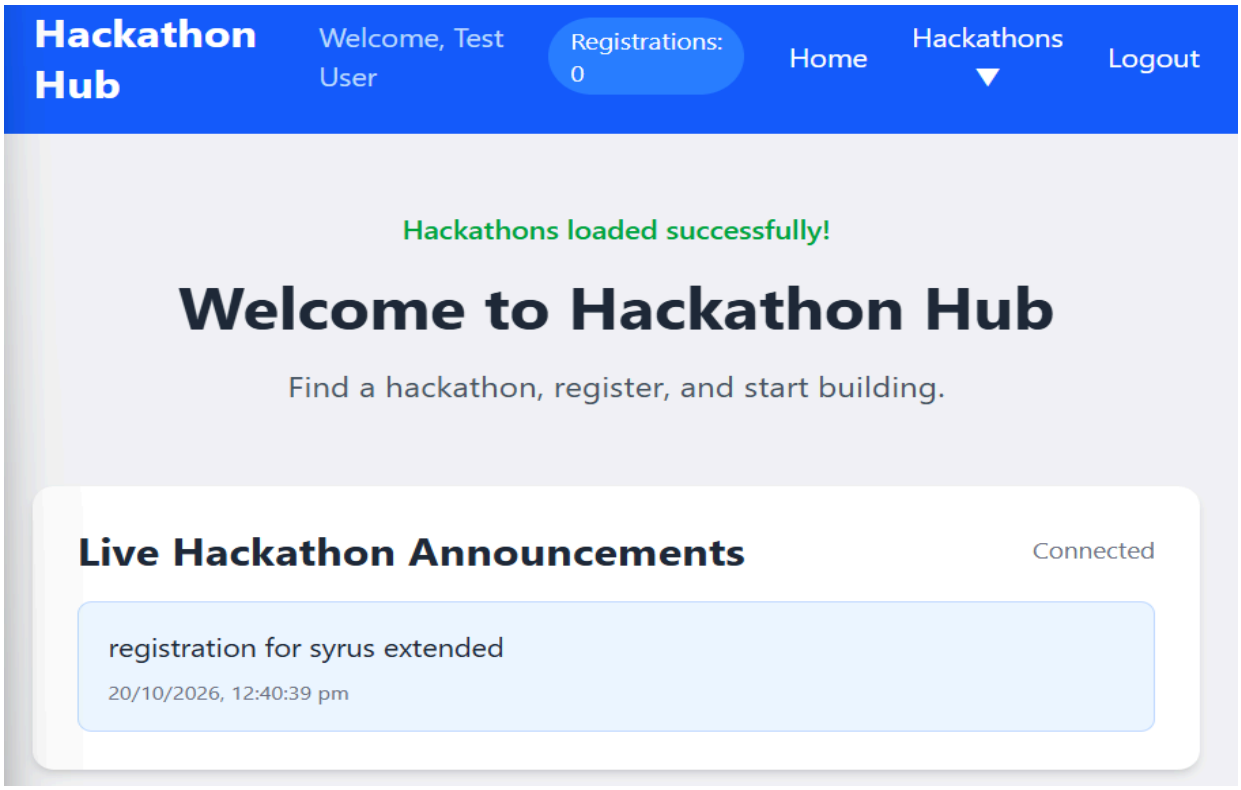
1. Admin Publishing Announcement

Use your current screenshot showing the Admin logged in, the Admin: Publish Announcement form, and the success message after publishing.



2. User Receiving Announcement

Use the right side of your current screenshot showing the regular User logged in and the announcement displayed under Live Hackathon Announcements.



3. Real-Time Communication in Both Windows: Use the full screenshot showing both browser windows side by side, with the same announcement visible in both and the connection status showing Connected.

